

Setor de Educação Profissional e Tecnológica Tecnologia em Análise e Desenvolvimento de Sistemas

DS141 - Estruturas de Dados II

04 - Busca

Prof. Alex Kutzke

Maio de 2021

Slides adaptados do material produzido
pelo Prof. Rodrigo Minetto

Roteiro

Introdução

Busca sequencial

Busca binária

Busca

Motivação: a operação de **busca** por uma **palavra-chave** ou **valor** é a base de muitas aplicações da computação, seja em um motor de busca na internet ou em um sistema comercial.

Busca

- O problema : uma coleção de elementos, onde cada elemento possui um identificador e uma chave para busca;
- Saída : o índice na coleção que possui a mesma chave procurada ou identificar que tal chave não existe na coleção.

Busca

Em nossos exemplos usaremos um vetor de inteiros como coleção. No entanto, podemos utilizar qualquer coleção cujos elementos possam ser comparados.

Busca

A forma mais simples de fazer uma busca em um vetor consistem em percorrer o vetor, elemento a elemento, para verificar se o elemento de interesse é igual a um dos elementos do vetor.

Exemplo Busca sequencial

- Elemento desejado: 66.

0	1	2	3	4	5	6	7
22	99	66	33	11	00	77	55

- Comparação

Exemplo Busca sequencial

- Elemento desejado: 66.

0	1	2	3	4	5	6	7
22	99	66	33	11	00	77	55

- Comparação

Exemplo Busca sequencial

- Elemento desejado: 66.

0	1	2	3	4	5	6	7
22	99	66	33	11	00	77	55

- Comparação

Exemplo Busca sequencial

- Elemento desejado: 66.

0	1	2	3	4	5	6	7
22	99	66	33	11	00	77	55

- Comparação

Exemplo Busca sequencial

- Elemento desejado: 66.

0	1	2	3	4	5	6	7
22	99	66	33	11	00	77	55

- Comparação;
- Elemento encontrado na posição 3.

Exemplo Busca sequencial

- Elemento desejado: 44.

0	1	2	3	4	5	6	7
22	99	66	33	11	00	77	55

- Comparação;

Exemplo Busca sequencial

- Elemento desejado: 44.

0	1	2	3	4	5	6	7
22	99	66	33	11	00	77	55

- Comparação;

Exemplo Busca sequencial

- Elemento desejado: 44.

0	1	2	3	4	5	6	7
22	99	66	33	11	00	77	55

- Comparação;

Exemplo Busca sequencial

- Elemento desejado: 44.

0	1	2	3	4	5	6	7
22	99	66	33	11	00	77	55

- Comparação;

Exemplo Busca sequencial

- Elemento desejado: 44.

0	1	2	3	4	5	6	7
22	99	66	33	11	00	77	55

- Comparação;

Exemplo Busca sequencial

- Elemento desejado: 44.

0	1	2	3	4	5	6	7
22	99	66	33	11	00	77	55

- Comparação;

Exemplo Busca sequencial

- Elemento desejado: 44.

0	1	2	3	4	5	6	7
22	99	66	33	11	00	77	55

- Comparação;

Exemplo Busca sequencial

- Elemento desejado: 44.

0	1	2	3	4	5	6	7
22	99	66	33	11	00	77	55

- Comparação;

Exemplo Busca sequencial

- Elemento desejado: 44.

0	1	2	3	4	5	6	7
22	99	66	33	11	00	77	55

- Comparação;

Exemplo Busca sequencial

- Elemento desejado: 44.

0	1	2	3	4	5	6	7
22	99	66	33	11	00	77	55

- Comparação;
- Não encontrou (posição: -1).

Busca binária

Em diversas aplicações reais, existe a necessidade de algoritmos eficientes. **Seria possível melhorar a eficiência do algoritmo de busca sequencial?**

Busca binária

Infelizmente, se os **elementos** estiverem **armazenados** em uma **ordem aleatória** no vetor, **não** temos como melhorar o algoritmo de busca, pois precisamos verificar todos os elementos.

Busca binária

No entanto, se os elementos do vetor estiverem ordenados, podemos aplicar um algoritmo mais eficiente para realizar a busca. Trata-se do algoritmo **Busca Binária**.

Busca binária

Contexto:

Jogo “adivinha o número”: pense em um número e diga o intervalo em que ele está. Para cada adivinhação de um oponente, apenas uma das três respostas podem ser dadas: **o número é maior, o número é menor ou acertou**. O vencedor é aquele que acertar o número com menos adivinhações.

Busca binária - conceito

1. Testar o elemento que buscamos com o valor armazenado no **meio** do vetor;
2. Se o elemento que buscamos for **menor** que o elemento do meio, sabemos que, se o elemento estiver presente no vetor, ele estará na primeira parte do vetor.
3. Se for **maior**, estará na segunda parte do vetor;
4. Se for **igual**, o elemento foi encontrado;
5. Se concluirmos que o elemento está em uma das partes do vetor, repetimos o procedimento descrito considerando apenas a parte restante.

Exemplo busca binária

- Elemento desejado: 55.

0	1	2	3	4	5	6	7
00	11	22	33	55	66	77	99

- Comparação;
- Intervalo considerado (início e fim);

Exemplo busca binária

- Elemento desejado: 55.

0	1	2	3	4	5	6	7
00	11	22	33	55	66	77	99
i							f

- Comparação;
- Intervalo considerado (início e fim);
- $\frac{(1 + 8)}{2} = 4$;

Exemplo busca binária

- Elemento desejado: 55.

0	1	2	3	4	5	6	7
00	11	22	33	55	66	77	99
				i			f

- Comparação;
- Intervalo considerado (início e fim);
- $\frac{(5 + 8)}{2} = 6;$

Exemplo busca binária

- Elemento desejado: 55.

0	1	2	3	4	5	6	7
00	11	22	33	55	66	77	99

i f

- Comparação;
- Intervalo considerado (início e fim);
- $\frac{(5 + 5)}{2} = 5;$

Exemplo busca binária

- Elemento desejado: 55.

0	1	2	3	4	5	6	7
00	11	22	33	55	66	77	99

i f

- Comparação;
- Intervalo considerado (início e fim);
- Elemento encontrado na posição 5;

Exemplo busca binária

- Elemento desejado: 15.

0	1	2	3	4	5	6	7
00	11	22	33	55	66	77	99

- Comparação;
- Intervalo considerado (início e fim);

Exemplo busca binária

- Elemento desejado: 15.

0	1	2	3	4	5	6	7
00	11	22	33	55	66	77	99
i							f

- Comparação;
- Intervalo considerado (início e fim);
- $\frac{(1 + 8)}{2} = 4;$

Exemplo busca binária

- Elemento desejado: 15.

0	1	2	3	4	5	6	7
00	11	22	33	55	66	77	99
i		f					

- Comparação;
- Intervalo considerado (início e fim);
- $\frac{(1 + 3)}{2} = 2$;

Exemplo busca binária

- Elemento desejado: 15.

0	1	2	3	4	5	6	7
00	11	22	33	55	66	77	99

i f

- Comparação;
- Intervalo considerado (início e fim);
- $\frac{(3 + 3)}{2} = 3$;

Exemplo busca binária

- Elemento desejado: 15.

0	1	2	3	4	5	6	7
00	11	22	33	55	66	77	99

i f

- Comparação;
- Intervalo considerado (início e fim);
- Não encontrou (posição -1);

Busca Sequencial - Análise

```
1  int busca(int * v, int n, int x){
2      for(i=0; i<n; i++)
3          if(v[i] == x)
4              return(i)
5      return(-1);
6  }
```

Quantas comparações o algoritmo Busca Sequencial realiza?

Busca Sequencial - Análise

```
1  int busca(int * v, int n, int x){
2      for(i=0; i<n; i++)
3          if(v[i] == x)
4              return(i)
5      return(-1);
6  }
```

Quantas comparações o algoritmo Busca Sequencial realiza?

Busca Sequencial - Análise

```
1  int busca(int * v, int n, int x){
2      for(i=0; i<n; i++)
3          if(v[i] == x)
4              return(i)
5      return(-1);
6  }
```

Quantas comparações o algoritmo Busca Sequencial realiza?

- Melhor caso?
- Pior caso?
- Caso médio?

Busca Sequencial - Análise

```
1  int busca(int * v, int n, int x){
2      for(i=0; i<n; i++)
3          if(v[i] == x)
4              return(i)
5      return(-1);
6  }
```

Quantas comparações o algoritmo Busca Sequencial realiza?

- Melhor caso? = 1
- Pior caso?
- Caso médio?

Busca Sequencial - Análise

```
1  int busca(int * v, int n, int x){  
2      for(i=0; i<n; i++)  
3          if(v[i] == x)  
4              return(i)  
5      return(-1);  
6  }
```

Quantas comparações o algoritmo Busca Sequencial realiza?

- Melhor caso? = 1
- Pior caso? = $\sum_{i=0}^n 1 = n$
- Caso médio?

Busca Sequencial - Análise

```
1  int busca(int * v, int n, int x){  
2      for(i=0; i<n; i++)  
3          if(v[i] == x)  
4              return(i)  
5      return(-1);  
6  }
```

Quantas comparações o algoritmo Busca Sequencial realiza?

- Melhor caso? = 1
- Pior caso? = $\sum_{i=0}^n 1 = n$
- Caso médio? = $\frac{n}{2}$

Busca Binária

```
1  int busca_binaria(int v[], int n, int elem){
2      l = 0; r = n-1;
3
4      while(l <= r){
5          m = (l+r)/2;
6          if(v[m] == elem) return(m);
7          else if(v[m] < elem) l = m + 1;
8          else if(v[m] > elem) r = m - 1;
9      }
10     return(-1);
11 }
```

Busca Binária Recursiva

```
1  int busca_binaria_recur(int v[], int n, int elem, int l, int r){
2      int m = (l+r)/2;
3
4      if(l > r)
5          return(-1);
6
7      if(v[m] == elem)
8          return(m);
9      else if(v[m] < elem)
10         return(busca_binaria_recur(v,n,elem,m+1,r));
11     else if(v[m] > elem)
12         return(busca_binaria_recur(v,n,elem,l,m-1));
13 }
```

Busca Binária - Análise

Dado um inteiro n , quantas vezes é possível dividir n ao meio, até que o seu valor seja 1 ou fique abaixo disso?

$$\frac{n}{2}, \frac{n}{4}, \frac{n}{8}, \dots, \frac{n}{2^k}$$

Busca Binária - Análise

$$\frac{n}{2}, \frac{n}{4}, \frac{n}{8}, \dots, \frac{n}{2^k}$$

Para achar o valor de k (número máximo de divisões):

$$\frac{n}{2^k} = 1$$

$$k = \log_2 n$$

$$k = \lceil \log_2 n \rceil$$

$$\sum_{i=1}^{\lceil \log_2 n \rceil} 1 = \log_2 n$$